

AsymptoticAnalysis

Discarded-tail reality,
condition growth, and
negative-Lerch enclosures

An incremental review for Wolfram 15 and Mathics3

Prepared for Vladimir Reshetnikov

10 September 2026

Reviewed repository `VladimirReshetnikov/Asymptotic`

Pinned revision

`efa1aeec4845a9c35e140963a0333d0c9ec33b05`

Commit timestamp 10 September 2026, 19:09:04 UTC

Established here

Two public Wolfram 15 reproductions; focused in-memory candidate checks; exact counterexample proofs; independent rational-reference tests.

Not claimed

Mathics execution, full upstream-suite acceptance, an installed patched release, or native timing improvements.

Abstract

This incremental audit isolates two defects not reissued from the inspected six-wave review baseline. First, a public operation chain hides a complex source coefficient beyond the computed jet, amplifies the discarded tail, and sends the resulting pure remainder through sine or cosine. The composite arithmetic fallback proves that the finite approximation is real, but then uses a global real-line Lipschitz bound for a complete function that need not be real. Wolfram 15 returns an algebraic error envelope where the true error grows exponentially. Second, repeated self-addition of a one-block expansion triples retained assumptions and target-domain predicates at each step, because condition combination occurs both during alignment and again during finalization. Both defects are reproduced through public calls; three narrowly scoped source replacements were exercised in memory on the complete retrieved standalone.

A separate development contribution supplies a proved, nested rational enclosure for the Lerch transcendent at negative arguments, including the endpoint $z = -1$. It uses the classical Euler transformation with positive finite differences, rather than absolute moments of an alternating source. Independent Python and Wolfram Language reference implementations accompany the proof. The report records exact source scope, nearby prior findings, actual versus unexecuted tests, candidate limitations, and a focused integration plan. Mathics behavior remains an explicit acceptance obligation, not an inference from shared source.

Contents

1	Executive assessment	3
1.1	What actually ran	3
2	Scope, source grounding, and review discipline	4
2.1	Snapshot and inspected surface	4
2.2	Excluding earlier findings	4
2.3	Evidence classes and failed infrastructure attempts	5
3	N01: a real finite part does not make the omitted tail real	5
3.1	Public reproduction and observed output	5
3.2	Exact counterexample	6
3.3	Source-level failure chain	7
3.4	Minimal repair and its proof	7
3.5	Native candidate observations	8
3.6	Limitations and a precise development path	8
4	N02: duplicated conditions grow while the coefficient jet stays small	9
4.1	Public experiment	9
4.2	Why the recurrence is exactly three in this path	9
4.3	Impact without overstating a benchmark	10
4.4	Candidate: structural idempotence, not assumption erasure	10
4.5	Native candidate result and acceptance criteria	10
5	E01: a negative-Lerch enclosure provider with positive transformed terms	11

5.1	Why this is a distinct development opportunity	11
5.2	Positive differences and the transformed expansion	11
5.3	Nested, explicit intervals	12
5.4	Exact reference implementation and resource model	13
5.5	Concrete endpoint and near-endpoint examples	14
5.6	Independent validation and its limits	14
5.7	Integration without conflating precision contracts	15
6	Cross-runtime assessment and focused acceptance plan	15
6.1	What can and cannot be transferred from Wolfram to Mathics	15
6.2	Acceptance matrix	16
6.3	A useful classification of new test results	16
7	Recommendations and development priorities	17
7.1	First: close the invalid trigonometric fallback	17
7.2	Second: make condition transport structurally idempotent	17
7.3	Third: add the bounded negative-Lerch provider	17
7.4	Decision gates rather than unsupported completion claims	17
8	Artifacts and reproduction	18
8.1	Archive contents	18
8.2	Independent tests	18
8.3	Review-only candidate patch generation	18
8.4	Native observation and desired-contract tests	19
8.5	Building the article	19
9	Conclusion	19
A	Novelty crosswalk	20
B	Source coverage record	20
C	Evidence inventory	22

1 Executive assessment

The useful additional review surface is not another general list of missing asymptotic features. The repository already has six waves of external reviews and a development status register. This report therefore concentrates on three specific contributions with executable artifacts and explicit novelty boundaries. The reviewed object is the pinned snapshot above, not an unspecified future state of `main`. Source links in the bibliography identify that same snapshot.[\[1, 2, 3\]](#)

ID	Priority	Contribution	Evidence
N01	High	Incorrect sine/cosine error transport after amplification of an omitted complex tail.	Public Wolfram 15 reproduction and exact counterexample.
N02	Medium	Exponential growth of persisted conditions during repeated one-block arithmetic.	Public counts through depth five; source recurrence.
E01	Development	Negative-Lerch interval provider based on positive Euler differences, including $z = -1$.	Proof; exact Python tests; 64 Wolfram reference combinations.

Table 1: Additional findings and development work. Priority describes this audit’s assessment, not an upstream issue status.

N01 deserves the first repair because it produces a plausible **GeneralizedSeries** with a mathematically false magnitude claim. It is not merely a conservative refusal, an unsupported function left unevaluated, or a correct complex result with inconsistent labeling. A refusal is a suitable minimal repair: the package need not implement a complete complex transseries algebra to stop asserting an unjustified real-line theorem.

N02 is a structural performance problem with an especially clean witness. The finite expression remains $2^na x$ and the number of coefficient blocks remains one, while a single logical predicate occurs 3^n times. The measured quantity is expression structure, not wall-clock time. The proposed fix preserves the mathematical hypotheses and removes their duplication in the tested path.

E01 is deliberately separated from the bug findings. It is a useful provider design with a complete proof and exact arithmetic implementation, not a claim that the existing package already computes the proposed intervals. Classical Euler acceleration is not new; the contribution is its tailored enclosure contract, endpoint coverage, independent implementation, and integration specification for the current negative-Lerch use case.[\[18, 8\]](#)

1.1 What actually ran

The native service reported:

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

The successful package experiments imported the entire pinned root standalone as text and evaluated that string. Each successful baseline import inspected here had `StringLength` 692380. The two findings and the focused patched controls below are actual native observations, preserved as transcribed records with their inputs in the archive. The canonical-module patch emitter was separately exercised with synthetic source fixtures; this is a different evidence population.

The independent Python suite passed 17 test methods, including a 700-case exact-rational Lerch grid, 84 comparisons with an independently bounded logarithmic reference, and 45 overlaps with

alternating-series reference intervals. The independent Wolfram reference passed 64 parameter combinations. These numbers must not be added together and described as upstream package tests. The two-test WLT acceptance file and file-based native wrappers are supplied but were not separately executed as complete suites.

No Mathics kernel ran. No full upstream suite ran. No installed candidate build or native performance benchmark ran. Those limits do not weaken the exact counterexample to a conjecture, but they do limit the scope of compatibility and release-readiness claims.

2 Scope, source grounding, and review discipline

2.1 Snapshot and inspected surface

The source was inspected through commit-pinned GitHub reads. A complete local checkout was not available. Native successful experiments independently obtained the pinned standalone; that execution does not imply that every line in the distribution was manually reviewed. The inspected areas include the main jet engine, ordinary and composite arithmetic, several inverse and certificate paths, Fourier and reciprocal-log operations, Gamma-related normalization, the Dirichlet special-function provider, and selected Mathics compatibility modules. Appendix B records module-level coverage and its limits.[4, 5, 6, 7, 8, 10]

The most important architectural boundary for this audit is the difference between a finite expression and what its remainder permits. An object may retain

$$f(x) = e(x) + \mathcal{O}(R(x))$$

on a recorded approach without establishing that f is real merely because e is real. Likewise, condition metadata can be logically unchanged while its expression tree grows dramatically. Both boundaries cross module interfaces, so isolated coefficient tests can miss the failure.

The package also has specialized representations and exact interval certificates. This report does not collapse them into one contract. A formal finite-model residual, an asymptotic magnitude bound, a derivative regularity assertion, and a quantitative closed interval are distinct evidence types. N01 concerns an asymptotic magnitude assertion; N02 concerns stored predicates; E01 constructs a quantitative forward interval independently of the package's inverse-certificate machinery.[5, 7, 9]

2.2 Excluding earlier findings

The exclusion baseline comprised the umbrella external-review index, all six wave indexes, the first 380 lines of the development status register, and the closest original report READMEs. In particular, reviews 11, 37, 42, 47, and 49 were consulted directly for their nearby mechanisms. This was not a new cover-to-cover reading of every retained or retired article, PDF, code artifact, and appendix. The novelty claim is therefore a specific mechanism-level contribution relative to that inspected baseline, not a formal exhaustive text-search guarantee.[2, 3, 13, 14, 15, 16, 17]

This qualification matters because several tempting leads are already represented in the archive. The present report does not reissue the complex-modulus sign shortcut, the derivative regularity of absolute value, generic coefficient-realness inconsistency, mutually recursive Mathics proof-search duplication, or positive-parameter Lerch enclosures as new findings. Section A explains exactly where the new mechanisms differ.

2.3 Evidence classes and failed infrastructure attempts

The report uses four practical evidence classes. A *native observation* is an actual result returned by the Wolfram service for a recorded input. A *source trace* identifies the functions that explain it. A *mathematical proof* establishes the relevant contract violation or enclosure independently of evaluator output. A *candidate acceptance obligation* is a test that should be run during integration but was not necessarily executed here.

Some remote calls failed with service/network errors. Some direct URL `Get` attempts received incomplete source streams and produced syntax errors at varying positions. Those attempts are excluded from package findings and test counts. Complete-text import followed by `Get[StringToStream[src]]` succeeded for the reported package experiments. The service was stateless between calls, so each successful experiment loaded its own source. Public package symbols were fully qualified in the recorded successful inputs to avoid parsing them into `Global`` before the package loaded.

These operational details are recorded so that a later reader can distinguish an upstream defect from an unsuccessful transport or an authoring mistake. They are not proposed as new repository defects.

3 N01: a real finite part does not make the omitted tail real

3.1 Public reproduction and observed output

Run the following in a fresh Wolfram kernel after loading the pinned package. The witness constructs its objects through public operations; it does not forge an association or invoke a private remainder helper.

```
s = AsymptoticAnalysis`AsymptoticExpansion[
  1/(1-a x^3), {x,0,1},
  Assumptions -> a^2 == -1, "Backend" -> "Package"];
t = AsymptoticAnalysis`SeriesMultiply[
  AsymptoticAnalysis`SeriesAdd[s,-1], x^-4];
{Sin[t], Cos[t]}
```

Table 2 lists selected actual fields. The original source assumption is $a^2 = -1$ and the target domain is $x > 0$; the latter objects contain duplicated copies of these predicates, discussed separately in N02.

Object	Kind / scale	Finite expression	Returned remainder
<code>s</code>	Forward	1	<code>PowerLogRemainder[x,2,0]</code>
<code>t</code>	Derived / PowerLog	0	<code>PowerLogRemainder[x,-2,0]</code>
<code>Sin[t]</code>	Derived / Composite	0	<code>PowerLogRemainder[x,-2,0]</code>
<code>Cos[t]</code>	Derived / Composite	1	<code>PowerLogRemainder[x,-2,0]</code>

Table 2: Native baseline observations. The final two rows assert errors of order x^{-2} as $x \rightarrow 0^+$.

The exact source is deliberately outside a purely real-valued source class. That is not a request that the package promise unrestricted complex support. It is a test of admission and subsequent

error transport: the package accepts the source, retains its defining function and assumptions, and returns a magnitude contract without rejecting the missing real-germ hypothesis. The defect can be repaired conservatively by refusal. It cannot be dismissed as an arbitrary malformed object, because the accepted object and all subsequent operations are public results.

The cubic source term is important. The ordinary forward engine starts this request with a working order above the user cutoff; the first possibly imaginary coefficient is still beyond that computed jet. Merely checking all *computed* rows for realness therefore does not encounter it. The returned $\mathcal{O}(x^2)$ is conservative for the true $\mathcal{O}(x^3)$ source correction. The subtraction and multiplication then faithfully turn that uncertainty into $\mathcal{O}(x^{-2})$. It is the final trigonometric rule that becomes false.[4, 5, 6, 7]

3.2 Exact counterexample

Choose $a = i$, which satisfies the retained assumption. The exact function represented by $\mathfrak{t}$ is

$$g(x) = x^{-4} \left(\frac{1}{1 - ix^3} - 1 \right) = \frac{i}{x(1 - ix^3)} \quad (1)$$

$$= -\frac{x^2}{1 + x^6} + i \frac{1}{x(1 + x^6)} = u(x) + iv(x). \quad (2)$$

The package's $g(x) = 0 + \mathcal{O}(x^{-2})$ is valid: in fact $|g(x)|$ is asymptotic to x^{-1} . Thus this example does not rely on an already false input envelope.

For real u, v , direct expansion of the sine and cosine gives

$$|\sin(u + iv)|^2 = \sin^2 u + \sinh^2 v, \quad (3)$$

$$|\cos(u + iv)|^2 = \cos^2 u + \sinh^2 v. \quad (4)$$

Consequently,

$$|\sin g(x)| \geq \sinh v(x), \quad |\cos g(x) - 1| \geq \sinh v(x) - 1.$$

These identities are consistent with the complex sine definition; the argument below is an exact elementary derivation, not a numerical comparison with a special-function evaluator.[20]

Proposition 3.1. *Neither of the two trigonometric remainder assertions in Table 2 is valid for the retained source family.*

Proof. It is enough to exhibit one permitted parameter specialization and one sequence on the recorded approach. Set $a = i$ and $x = 1/n$, with positive integers n . From (2),

$$v(1/n) = \frac{n}{1 + n^{-6}} \geq \frac{n}{2}.$$

The positive Taylor coefficients of $\sinh$ imply

$$n^{-2} |\sin g(1/n)| \geq n^{-2} \sinh(n/2) \geq \frac{n^5}{2^7 7!} = \frac{n^5}{645120} \longrightarrow \infty.$$

Similarly,

$$n^{-2} |\cos g(1/n) - 1| \geq \frac{n^5}{645120} - \frac{1}{n^2} \longrightarrow \infty.$$

An $\mathcal{O}(x^{-2})$ error would make both normalized quantities bounded for sufficiently small positive x . Hence both returned claims fail. The actual growth is exponential in $1/x$; the displayed polynomial lower bounds already suffice to prove the violation. $\square$

This proof does not assume a particular branch chosen for a by a solver. The other specialization $a = -i$ conjugates the function and yields the same magnitude contradiction. It also does not depend on a floating-point threshold, an estimated leading coefficient, or an interpretation of a numerical plot.

3.3 Source-level failure chain

The failure is explained by the interaction of four layers. In the main engine, `forwardJet` chooses working precision and `makeForwardObject` validates the coefficient rows it receives. That is a check of the available finite expansion, not a proof about every discarded source term.[4]

In ordinary series arithmetic, the source correction is discarded, the constant part cancels, and multiplication by x^{-4} shifts the uncertainty. No incorrect estimate is needed at this stage. `SeriesAdd` and `SeriesMultiply` preserve the finite approximation and its available magnitude information.[5, 6]

When sine or cosine is applied, the ordinary observable path cannot use a finite-limit Taylor argument with the nonvanishing error precision. `seriesArithmeticUnary` admits a conservative composite fallback for supported unary heads. This fallback is a legitimate architectural option; it is not itself the bug.[6]

Finally, `seriesEnvelopeUnary` proves that the finite `Expression` is eventually real, then groups `Abs`, `Sin`, and `Cos` under a global real-line one-Lipschitz statement. In this example the finite expression is exactly zero, so its realness test is immediate. The test says nothing about the imaginary component of g . The stored source and its remainder do not supply the missing complete-real-germ proof.[7]

The distinction between these three heads is decisive. Complex modulus satisfies

$$||z| - |w|| \leq |z - w| \quad (z, w \in \mathbb{C}).$$

Sine and cosine do not have a corresponding global bound on $\mathbb{C}$: their derivatives grow along imaginary directions. A shared implementation branch based only on a common real-line statement conceals that difference.

3.4 Minimal repair and its proof

The supplied N01 edit is deliberately conservative. Before transporting a nonzero remainder through sine or cosine, it requires the existing envelope-limit mechanism to prove

$$R(x) \longrightarrow 0.$$

If it cannot, the operation returns `Failure["UnprovedRealRemainder", ...]`. Modulus does not receive this additional requirement. The candidate changes only the relevant composite unary branch; it is not a general rewrite of source admission or a solution to every complex-modulus issue previously reviewed.

The additional requirement has a simple independent justification.

Proposition 3.2. *Let $e(x)$ be real on the approach, let $\delta(x)$ be complex, and suppose $\delta(x) = \mathcal{O}(R(x))$ for a nonnegative $R(x) \rightarrow 0$. Then*

$$\sin(e + \delta) - \sin e = \mathcal{O}(R), \quad \cos(e + \delta) - \cos e = \mathcal{O}(R).$$

No boundedness assumption on the real finite part e is needed.

Proof. Integrate the complex derivative along the straight segment from e to $e + \delta$. Since e is real,

$$|\cos(e + t\delta)|, |\sin(e + t\delta)| \leq e^{|\operatorname{Im}(t\delta)|} \leq e^{|\delta|} \quad (0 \leq t \leq 1).$$

Both differences have magnitude at most $|\delta|e^{|\delta|}$. Because $\delta \rightarrow 0$, this is $\mathcal{O}(R)$. $\square$

The candidate also corrects the proof metadata. For the local complex argument, retaining `LipschitzConstant` equal to one would be inaccurate: even $|\sin(it)| = \sinh t$ exceeds t for positive t . The patched sine/cosine result instead records a local complex-strip bound and the vanishing-envelope limit, without asserting a global numerical constant. The modulus case retains its genuine global constant one.

3.5 Native candidate observations

All three anchors for the combined N01/N02 candidate were found exactly once in the full imported standalone. The edited string was evaluated in Wolfram 15. On that source, the two unsafe trigonometric requests both returned the intended `UnprovedRealRemainder` failure. Sine and cosine of the ordinary real expansion of $1/(1-x)$ to cutoff three still returned `GeneralizedSeries`. Applying `Abs` to the amplified witness still returned $0 + \mathcal{O}(x^{-2})$, which is a valid conservative modulus bound.

These are focused acceptance observations, not a claim that all unary routes, all public options, every special representation, or the full regression suite pass. The complete replacement strings and native input are shipped; the observed output is recorded in `evidence/wolfram_observations.md`.

3.6 Limitations and a precise development path

A vanishing-envelope requirement can reject valid real problems with large uncertain remainders. That loss is intentional in the minimal fix. For a proved real complete function, sine and cosine are globally one-Lipschitz even when the magnitude of its difference from the finite approximation grows. Recovering that useful behavior requires an independent proof of the represented germ's value domain, not another check of its displayed coefficients.

A more precise follow-on interface could retain a capability such as *complete represented germ is eventually real*, supported by a source proof or by sound transport from operands. This is a design proposal, not a property already present or validated. It should have a stated approach and assumptions, and it should survive an operation only when the operation preserves realness on that domain. A formal derivative contract or a row-realness flag is not a substitute.

There is also a safe intermediate extension: an eventually bounded envelope, rather than a vanishing one, suffices for the local strip argument with an existential constant. The narrow candidate uses the existing zero-limit predicate to minimize the change. An extension to arbitrary bounded envelopes should prove boundedness explicitly and distinguish it from a finite numerical derivative certificate.

For unbounded complex uncertainty, writing $\mathcal{O}(e^R)$ is not automatically a repair. If $\delta = \mathcal{O}(R)$, the unknown constant can move into the exponential, producing e^{CR} rather than $\mathcal{O}(e^R)$. Either an explicit bound on the imaginary perturbation or a scale family accommodating that constant is required. Failing closed until such evidence exists is preferable to replacing one false envelope with another.

4 N02: duplicated conditions grow while the coefficient jet stays small

4.1 Public experiment

Construct the exact one-block expansion

```
s = AsymptoticAnalysis'AsymptoticExpansion[
  a x, {x,0,2}, Assumptions -> a > 0,
  "Backend" -> "Package"];
(* Repeat: s = AsymptoticAnalysis'SeriesAdd[s,s] *)
```

The recorded baseline measured the number of blocks, `LeafCount` of the assumptions and target domain, and occurrences of the predicate $a > 0$ before each update. Table 3 is the native result, not a simulated expression-growth model.

Depth	Blocks	Copies of $a > 0$	Assumption leaves	Domain leaves
0	1	1	3	3
1	1	3	10	10
2	1	9	28	28
3	1	27	82	82
4	1	81	244	244
5	1	243	730	730

Table 3: Baseline structural growth in public self-addition. The corresponding finite expression is $2^n ax$.

The domain shows the same pattern with $x > 0$. Neither the mathematical parameter restriction nor the target approach changes under self-addition. The input and output require the same two logical facts throughout.

4.2 Why the recurrence is exactly three in this path

`seriesAlign[a,b]` already combines the two operands' assumptions and domains while aligning the formal logarithm variable. In schematic notation it replaces the second operand's assumption by

$$B' = A \wedge B.$$

Later, `seriesBinary` finalizes its result with the first operand's assumption and the *already aligned* second assumption:

$$A \wedge B' = A \wedge (A \wedge B).$$

The same construction is used for the domain. Both source locations are in `SeriesOperations.wl.[5]`

For self-addition, suppose the current assumption expression contains C_n copies of the original predicate. The next aligned second operand contains $2C_n$ copies; finalization adds the first operand's C_n copies again. Therefore

$$C_0 = 1, \quad C_{n+1} = 3C_n, \quad C_n = 3^n.$$

For $n \geq 1$, the displayed flat conjunction in this witness has one **And** head and three leaves for each $a > 0$ predicate, explaining the observed count $1 + 3 \cdot 3^n$. At depth zero there is no conjunction head, so the count is three.

The source recurrence and the measurements agree through all six observed states. This is not an extrapolated claim about undocumented Wolfram simplifier internals: it is the expression structure actually retained by the package on these requests. Structural deletion of repeated conjuncts is semantically justified by idempotence of conjunction.^[21]

4.3 Impact without overstating a benchmark

The witness separates coefficient complexity from metadata complexity. Each result needs one coefficient block and a constant-size list of distinct hypotheses. The duplicated predicates can nevertheless increase the input presented to later simplification, domain checks, property inspection, serialization, and expression traversal.

It would be unjustified to convert these counts directly into an elapsed-time ratio or a physical-memory allocation ratio. The evaluator may share expression subtrees, short-circuit predicates, or canonicalize some inputs during later operations. None of that was benchmarked. The finding is that unnecessary exponentially growing logical syntax is generated and retained; the subsequent cost depends on its consumers.

The operation recipe can also retain its own operand history. Removing condition duplication does not eliminate that independent provenance growth and does not prove that an entire arithmetic expression DAG has constant memory. The proposed repair is intentionally limited to the observed assumptions/domain amplification.

4.4 Candidate: structural idempotence, not assumption erasure

The candidate introduces a small helper:

```
seriesStructuralAnd[conditions_List] := And @@ DeleteDuplicates[
  Flatten[(If[Head[#] === And, List @@ #, {#}] &) /@ conditions, 1]];
```

It is used for both assumption and domain joins in `seriesAlign` and in the final `seriesBinary` result. Existing top-level conjunctions are combined and structurally identical predicates are retained once. Other expressions are treated as whole predicates. The helper does not enter held subexpressions or perform a solver-driven logical minimization.

Using `FullSimplify[conditions, conditions]` would be a different and dangerous operation: it can return `True` precisely because the hypotheses were supplied as assumptions, thereby deleting restrictions from the result. The candidate instead applies the elementary identity $P \wedge P = P$ to already retained mathematical predicates. Distinct conditions remain distinct; this is not permission to specialize an object under a new ambient assumption context.

A further implementation refinement could calculate the combined predicates once and pass the same value to alignment and finalization. That may avoid small repeated construction work, but it is not necessary for the tested structural invariant. The supplied change is kept narrow so that its effect and proof obligation are easy to inspect.

4.5 Native candidate result and acceptance criteria

After applying the candidate to the full standalone string, the observed rows at depths zero through five were

```
{0,1,1,True}, {1,1,1,True}, {2,1,1,True},
{3,1,1,True}, {4,1,1,True}, {5,1,1,True}
```

Each row is `{depth, assumptionOccurrences, domainOccurrences, correctExpression}`. The final boolean checks exact structural equality of the normal expression with $2^n ax$. Thus the predicates remained present once and the finite expression remained correct throughout the tested sequence.

Integration should additionally test unequal operands with independent hypotheses, contradictory domains, shared versus distinct conjuncts, and target-coordinate alignment. The acceptance invariant is not that every logical formula has a globally minimal representation. It is that combining an operand with itself does not multiply structurally identical retained assumptions or domains, while every distinct hypothesis still survives.

The candidate is shared Wolfram Language code. Its Mathics behavior is not established by the native Wolfram run; in particular, host evaluation of conjunctions and association operations must be tested directly. This difference in evidence is retained in the intake record rather than hidden behind a generic portability label.

5 E01: a negative-Lerch enclosure provider with positive transformed terms

5.1 Why this is a distinct development opportunity

The inspected Dirichlet provider admits fixed real Lerch parameters with $|z| < 1$, builds a large-argument moment expansion, and obtains a quantitative bound by summing absolute geometric moments. Its proof is appropriate to that construction, but it suppresses the cancellation of a negative argument when it replaces z by $|z|$. The dedicated provider also excludes the endpoint $z = -1$.^[8]

This does not imply that no other package route or native simplification can evaluate any expression at $z = -1$. The narrow statement is about the source-admission condition and bound mechanism of the inspected dedicated provider. The proposed method is an additional forward-enclosure route, not a replacement for the existing moment expansion and not a universal Lerch implementation.

Review 49's nearby contribution concerns positive-parameter signed Lerch enclosures. The method here instead covers $z = -q$ for $0 \leq q \leq 1$ using a positive Euler-transformed series. It includes a boundary case excluded by the inspected defining-sum provider and has a geometric interval-width bound uniform in q on that interval.^[17]

The starting point is the defining series

$$\Phi(-q, s, a) = \sum_{j=0}^{\infty} \frac{(-q)^j}{(a+j)^s}, \quad 0 \leq q \leq 1, \quad s > 0, \quad a > 0. \quad (5)$$

At $q = 1$ it converges by the alternating-series criterion. The defining series and integral representations are classical; so is the Euler transformation used below. The bounds are derived explicitly to make the reference implementation independently auditable.^[19, 18]

5.2 Positive differences and the transformed expansion

Set

$$b_j = (a+j)^{-s}, \quad D_k = \sum_{j=0}^k (-1)^j \binom{k}{j} b_j, \quad r = \frac{q}{1+q}. \quad (6)$$

The sign convention makes D_k the positive alternating finite difference of a completely monotone sequence, rather than the usual unsigned forward difference with a possible factor $(-1)^k$.

Theorem 5.1 (Positive Euler representation). *For the parameters in (5),*

$$0 < D_{k+1} \leq D_k \leq a^{-s}, \quad \Phi(-q, s, a) = \frac{1}{1+q} \sum_{k=0}^{\infty} r^k D_k. \quad (7)$$

In particular, $0 \leq r \leq 1/2$ and the transformed series is absolutely convergent, including $q = 1$.

Proof. The elementary Laplace integral for a positive real power gives

$$b_j = \frac{1}{\Gamma(s)} \int_0^{\infty} t^{s-1} e^{-(a+j)t} dt.$$

Substitute this expression into the finite sum defining D_k and use the binomial theorem:

$$D_k = \frac{1}{\Gamma(s)} \int_0^{\infty} t^{s-1} e^{-at} (1 - e^{-t})^k dt. \quad (8)$$

Its integrand is positive for $t > 0$, and the factor $1 - e^{-t}$ lies strictly between zero and one. Positivity, monotonicity in k , and the bound by $D_0 = a^{-s}$ follow.

For $q < 1$, summing the original geometric factor inside the Laplace integral gives

$$\Phi(-q, s, a) = \frac{1}{\Gamma(s)} \int_0^{\infty} \frac{t^{s-1} e^{-at}}{1 + qe^{-t}} dt.$$

The identity

$$\frac{1}{1 + qe^{-t}} = \frac{1}{1+q} \frac{1}{1 - r(1 - e^{-t})} = \frac{1}{1+q} \sum_{k=0}^{\infty} r^k (1 - e^{-t})^k$$

has nonnegative summands. Monotone convergence and (8) yield the transformed formula. The denominator of the integral is at least one; the integrable function $t^{s-1} e^{-at}$ therefore also permits passage to $q = 1$. Equivalently, the limit agrees with the endpoint alternating series. Since $D_k \leq a^{-s}$ and $r \leq 1/2$, the transformed series is absolutely bounded by a geometric series throughout the stated domain. $\square$

5.3 Nested, explicit intervals

Define the K -term transformed partial sum, with $K \geq 0$, by

$$E_K = \frac{1}{1+q} \sum_{k=0}^{K-1} r^k D_k, \quad E_0 = 0.$$

Empty powers are interpreted in the combinatorial sense: $r^0 = 1$, including when $r = 0$. This convention is implemented explicitly in the Wolfram code, where the raw expression 0^0 must not be used as an ordinary numerical value.

Theorem 5.2 (Negative-Lerch enclosure). *Let*

$$L_K = E_K + \frac{r^K D_K}{1+q}, \quad U_K = E_K + r^K D_K. \quad (9)$$

Then

$$L_K \leq \Phi(-q, s, a) \leq U_K, \quad U_K - L_K = r^{K+1} D_K \leq \frac{a^{-s}}{2^{K+1}}. \quad (10)$$

The closed intervals $[L_K, U_K]$ are nested as K increases.

Proof. The omitted transformed tail is

$$T_K = \frac{1}{1+q} \sum_{k=K}^{\infty} r^k D_k.$$

Its first term is a lower bound. Monotonicity $D_k \leq D_K$ for $k \geq K$ and the geometric sum give

$$\frac{r^K D_K}{1+q} \leq T_K \leq \frac{r^K D_K}{(1+q)(1-r)} = r^K D_K.$$

This proves enclosure. Subtracting the endpoints gives

$$r^K D_K \left(1 - \frac{1}{1+q}\right) = r^{K+1} D_K,$$

and the universal width estimate follows from $D_K \leq a^{-s}$ and $r \leq 1/2$.

For nestedness, $L_K = E_{K+1}$ increases. The upper endpoints satisfy

$$U_K - U_{K+1} = r^{K+1}(D_K - D_{K+1}) \geq 0.$$

At $q = 0$ the two endpoints coincide with a^{-s} for every K , including $K = 0$, by the empty-power convention. $\square$

A convenient sufficient order for a requested positive absolute width ε is any nonnegative integer K with

$$2^{-(K+1)} a^{-s} \leq \varepsilon.$$

The actual width $r^{K+1} D_K$ is often much smaller. An adaptive implementation should test this exact width rather than confuse the sufficient estimate with the achieved result. A relative-error stopping rule would require a positive lower bound for the represented value; the positive L_K already supplies one, so a separately defined sufficient criterion can be used. Neither stopping rule changes the meaning of an asymptotic exponent cutoff in the existing package.

5.4 Exact reference implementation and resource model

The theorem allows all real $s > 0$. The shipped implementation deliberately narrows the executable domain to positive integer s and rational q, a , so all samples b_j and all interval endpoints are exact rationals. It does not approximate an irrational power and then label its result rationally certified.

The Python public entry is:

```
lerch_negative_interval(q, s, a, order=16)
```

It accepts integers or `fractions.Fraction` for q, a , rejects floats and booleans, requires $0 \leq q \leq 1$ and $a > 0$, and bounds $1 \leq s \leq 1000$ and $0 \leq K \leq 256$. It also imposes a 4096-bit input-rational size limit. These are explicit defensive limits of the reference, not a theorem about feasible arithmetic cost at the largest accepted combination.

The finite differences are formed by a shrinking table. Starting from

$$(b_0, b_1, \dots, b_K),$$

record the first entry, replace each adjacent pair (v_j, v_{j+1}) by $v_j - v_{j+1}$, and repeat until the row is empty. The recorded first entries are $D_0, \dots, D_K$. The implementation uses $\mathcal{O}(K^2)$

exact subtractions and $\mathcal{O}(K)$ live rational entries, in addition to the output list. Those are arithmetic-operation and entry counts, not bit-complexity bounds. Numerators and denominators can grow substantially.

Floating-point evaluation of this difference table would be a different algorithm with cancellation risk. The exact rational implementation does not justify a naive machine-real translation. A numerical production version should use independently bounded arithmetic or exploit (8) with a proved quadrature/bound method. The narrow integer/rational reference is valuable precisely because it does not need those additional analytic or rounding contracts.

The independent Wolfram entry is:

```
AsymptoticAudit'NegativeLerchInterval[q,s,a,k]
```

It returns an association containing lower and upper endpoints, width, partial sum, the frontier difference D_K , parameters, and a theorem identifier. It uses its own context and changes no `AsymptoticAnalysis'` or `System'` definitions. Its parameter/order limits match the principal rational/integer restrictions; the Python-only input bit-size guard is not advertised as present in the WL reference.

5.5 Concrete endpoint and near-endpoint examples

For $q = s = a = 1$, the original value is $\log 2$, and the difference coefficients simplify to $D_k = 1/(k+1)$. At $K = 8$, the native reference returned

$$\frac{447047}{645120} \leq \log 2 \leq \frac{447187}{645120}, \quad U_8 - L_8 = \frac{1}{4608}.$$

This endpoint example is intentionally modest in order: it is a transparent rational check of the theorem and the $q = 1$ path, not a claim of competitive high-precision logarithm evaluation. The Euler expansion for this classical alternating logarithmic series is a standard special case.^[18]

At $q = 9/10$, $s = 2$, $a = 100$, $K = 8$, the returned exact width was

$$\frac{22679550475318225809}{7679474155727325039983857353312740000} \approx 2.953 \times 10^{-18}. \quad (11)$$

This is an enclosure width for a fixed positive argument, not an experimentally fitted remainder constant and not an assertion that an asymptotic Taylor truncation has changed order.

5.6 Independent validation and its limits

The 700-case grid uses

$$\begin{aligned} q &\in \{0, 1/10, 1/2, 9/10, 1\}, & s &\in \{1, 2, 3, 5\}, \\ a &\in \{1/4, 1, 3/2, 10, 100\}, & K &\in \{0, 1, 2, 4, 8, 16, 32\}. \end{aligned}$$

It checks exact width, endpoint order, the universal bound, and nestedness between successive selected orders. Separate tests compare the difference table with the explicit binomial sum and verify positivity and monotonicity on the tested parameter grid. A JSON record preserves every enclosure.

An independent reference for $s = 1$ and positive integer $a = m$ is

$$\Phi(-q, 1, m) = (-q)^{-m} \left[-\log(1+q) - \sum_{j=1}^{m-1} \frac{(-q)^j}{j} \right], \quad q > 0. \quad (12)$$

The test implementation bounds the logarithm independently with the positive atanh series, using $u = q/(2 + q)$ and an explicit geometric tail. Across 84 cases, the resulting tight reference interval lay inside the Euler enclosure. This comparison does not reuse the Euler difference table.

A further 45 cases compare with an independently bounded alternating partial sum. They check *overlap*, not containment; wide endpoint alternating intervals can make overlap a weak test. The article’s proof, rather than a count of overlapping intervals, establishes universal validity.

The independent native Wolfram run checked 64 combinations: four q values, two s values, two a values, and four orders. All exact width and ordering predicates returned true; an approximate input returned **Failure**. This was the fully qualified arithmetic core evaluated through the connector. The packaged file loader and wrapper were not separately run, and Mathics execution remains unperformed.

5.7 Integration without conflating precision contracts

A practical integration would expose this as a quantitative negative-Lerch forward provider, initially on the exact-rational reference domain. Its result should identify the argument $-q$, parameters, interval endpoints, transformed order K , and the theorem supporting containment. It should not impersonate the existing inverse root certificate or attach a numerical certificate to a bare $\mathcal{O}$ statement without constants.

There are two distinct uses. For a fixed target argument a , the interval can provide a certified numerical reference independent of the asymptotic polynomial. For a large- a asymptotic result, subtracting its exact rational finite approximation from both interval endpoints yields a quantitative interval for that particular truncation error. This does not automatically alter the stored asymptotic exponent or logarithmic degree. If the finite approximation has irrational coefficients, subtraction requires their own enclosures.

The method also gives a concrete development path at $z = -1$ without an absolute-moment blowup. It does not cover $q > 1$, arbitrary complex z , negative or singular a , or nonpositive s under the theorem stated here. Those cases should not be admitted by extrapolating the reference’s name or output shape.

6 Cross-runtime assessment and focused acceptance plan

6.1 What can and cannot be transferred from Wolfram to Mathics

The inspected package uses shared WL source together with isolated Mathics compatibility helpers and late-loaded adapters. Those helpers address evaluation semantics, assumptions, numerical routines, and other host differences. The existence of that architecture does not establish that a particular public witness reaches the same branch in both evaluators.[\[4, 10, 11, 12\]](#)

For N01, Mathics might refuse the complex parameter condition earlier, retain a different working jet, or reach the same unsafe composite fallback. Only the last case reproduces the native wrong result; the first two need to be recorded as distinct behavior. The source-level missing hypothesis in the shared fallback is still a reason to repair it, but this report does not label the public Mathics path as reproduced.

For N02, host handling of **And**, associations, and simplification may affect the visible condition expression. The recurrence is observed in Wolfram and explained by shared source joins. It is not a measured Mathics allocation or timing curve. The structural idempotence patch should be checked on the selected Mathics version with the same predicate-count invariant.

For E01, the exact reference avoids native **LerchPhi**, **PolyLog**, and root-finding facilities. That

reduces dependencies but does not amount to a compatibility certificate: exact rational arithmetic, list operations, association construction, local scoping, and failure behavior still need a real Mathics run.

6.2 Acceptance matrix

Area	Executed here	Required integration check	Failure interpretation
N01 baseline	Public Wolfram sine/cosine witness and exact proof.	Same source, cutoff, assumptions, and public sequence in Mathics.	Early conservative refusal is not the false-envelope bug.
N01 candidate	Both unsafe results refused; real and modulus controls preserved.	Real nonvanishing error; exact input; small complex error; all supported unary entry routes.	A new conservative refusal needs classification, not concealment.
N02 baseline	Six Wolfram states with 3^n predicate counts.	Exact counts on Mathics and mixed-condition operands.	Do not infer cost from block count alone.
N02 candidate	One copy of each predicate; exact $2^n ax$ through depth five.	Independent hypotheses and contradictory domains preserved.	Dropping a distinct hypothesis is a correctness regression.
E01	Python exact tests and 64 native reference cases.	File-loaded reference and endpoint cases on Mathics.	Unevaluated output is not a certified interval.
Release	No full upstream suite or installed build.	Regenerate standalone and run upstream integration validation.	Focused in-memory success is not release acceptance.

6.3 A useful classification of new test results

A future run should record the exact runtime version, repository revision, constructor result head, failure tag if any, finite expression, remainder, and retained assumptions/domain. A completed call that returns an unevaluated expression, a native formatting artifact, or `Failure` must not be counted as a successful expansion solely because it did not abort.

For the N01 family, the most informative progression is to vary the source exponent and the amplification exponent while keeping the first complex coefficient beyond the working precision. This tests the missing hypothesis, not one hard-coded polynomial. The deterministic cubic witness should remain as the smallest shipped regression because its expected behavior and proof are clear.

For N02, doubling an exact one-block source isolates metadata growth from asymptotic coefficient generation. Mixed conjunctions then test that the repair preserves semantics rather than merely shrinking expression size. Native elapsed-time measurements may be added later, but they must separate simplifier time, source replay, and provenance storage from the narrow condition-join cost.

These are acceptance tasks directly tied to the new findings, not a repetition of the repository's previously proposed general testing roadmap.

7 Recommendations and development priorities

7.1 First: close the invalid trigonometric fallback

Adopt the conservative N01 guard, or an equivalently justified complete-real-germ check, before broadening complex behavior. The key acceptance requirement is that a real displayed polynomial cannot authorize an unbounded complex sine/cosine error transport. Keep modulus separate because its global complex theorem really is stronger. Keep the proof metadata consistent with the theorem actually used.

A broader capability-based realness design should follow only with explicit source and transport proofs. Merely adding a new boolean field defaulted from coefficient realness would encode the same defect under a different name. The cubic witness must continue to be rejected or bounded correctly even if all retained coefficients remain real.

7.2 Second: make condition transport structurally idempotent

The N02 candidate is small and has a direct invariant. Apply it at the canonical module level, regenerate the distribution, and preserve tests for both idempotence and distinct-hypothesis retention. Do not replace duplicate elimination with simplification under the conditions themselves. Treat operation recipes as a separate provenance concern; this patch neither promises nor requires their wholesale redesign.

7.3 Third: add the bounded negative-Lerch provider

Integrate the exact-rational E01 reference behind a deliberately narrow admission predicate. Keep transformed-series order, achieved interval width, and the existing asymptotic cutoff as distinct user-facing quantities. Expose the $q = 1$ endpoint only with the positive-real a, s assumptions of the proof, and fail clearly outside the implemented arithmetic domain.

The uniform geometric width bound gives a transparent initial work estimate, while the exact achieved width gives the actual stop condition. The method is especially attractive as an independent checking route for negative-argument examples, because it avoids asking the same asymptotic moment engine to certify its own coefficients.

7.4 Decision gates rather than unsupported completion claims

The candidate edits have passed the small native observations documented here. They should enter the repository as reviewed candidates until the canonical rebuild and required upstream tests pass. The negative-Lerch reference has stronger independent mathematical coverage, but it is not integrated, so its result must not be advertised as a package capability yet. Mathics should receive its own observation and acceptance record rather than a status copied from Wolfram.

This ordering addresses one false mathematical claim, one avoidable structural growth mechanism, and one implementable new provider. It does not depend on restating any of the old general feature backlog.

8 Artifacts and reproduction

8.1 Archive contents

The article's complete editable source is `article.tex`; the compiled reading version is `article.pdf`. The archive contains independent Lerch implementations, executable Python checks, public native observation requests, two desired-contract WLT tests, three guarded source replacement records, a patch emitter, and actual evidence records. It does not contain a full upstream distribution, checksum files, Python caches, or separate font files.

The compact machine-readable intake is `findings.json`. The source and execution boundary is recorded in `evidence/audit_manifest.json`. The manually transcribed native record is `evidence/wolfram_observations.md`, alongside the actual successful connector input texts. These files identify which results were observed and which wrappers remain unexecuted.

8.2 Independent tests

From the extracted archive directory, using Python 3.10 or later:

```
python -S -m unittest discover -s tests -p 'test_*.py' -v
```

The suite uses only the standard library. It regenerates the mathematical grid and counterexample-bound records. Its patch tests use synthetic fixtures to check unique anchors, missing/duplicate anchors, selection, and output behavior. They are not represented as an actual canonical-source integration run.

For the WL reference, load the file in a fresh kernel and then the reference wrapper:

```
Get["/path/to/archive/code/NegativeLerchInterval.wl"];  
Get["/path/to/archive/tests/reference_wl.wl"];
```

The standalone reference package is independent of the audited repository. Its arithmetic was tested through the connector, while the file-based wrapper remains a reproduction artifact rather than an executed receipt.

8.3 Review-only candidate patch generation

Use a disposable local checkout of the pinned revision:

```
python patches/emit_candidate_patch.py /path/to/Asymptotic > candidate.patch
```

The emitter verifies Git HEAD unless explicitly overridden, reads the two canonical modules named in `edits.json`, requires a unique occurrence of each exact anchor, and prints a unified diff. It does not apply changes or write to the checkout. It also rejects a resolved source path escaping the selected root. It is not a full repository-cleanliness or security validator; unique anchors and a revision check do not prove that unrelated worktree changes are absent.

Use `-finding N01` or `-finding N02` for a single finding. The explicit `-allow-other-revision` option skips the Git pin check but retains the anchor checks. The anchors are LF text, so a CRLF-only source copy may require reviewed line-ending normalization. The archive does not silently normalize an unknown source and claim it matched exactly.

After reviewing and applying the changes in a separate worktree, run the upstream standalone

builder:

```
python validation/build_standalone.py
```

Testing an unchanged generated root distribution after editing only modular sources would not exercise the candidate. Conversely, this audit's in-memory replacement in the retrieved standalone does not prove that the canonical rebuild pipeline passes. Those two routes are intentionally distinguished.

8.4 Native observation and desired-contract tests

The supplied `tests/run_native.wls` uses a local path selected by `ASYMPTOTIC_REPO`. It prints observations and exits zero when the wrapper reaches its end; zero is not a claim that the known baseline defects are fixed. Confirm the checkout revision separately because this observation wrapper does not enforce the pin.

```
ASYMPTOTIC_REPO=/path/to/Asymptotic \
wolframscript -file tests/run_native.wls
```

For the two focused patched invariants:

```
Get["/path/to/patched/AsymptoticAnalysis.wl"];
TestReport["/path/to/archive/tests/patch_acceptance.wlt"]
```

The file should be evaluated in a fresh kernel without definitions for its symbols. It is a small desired-contract suite, not the complete upstream acceptance suite. The archive does not claim that these file-based commands were run on Mathics or as a Wolfram CLI process during the audit.

8.5 Building the article

```
sh build.sh
```

The source uses common pdfLaTeX packages, no shell escape, and no external bibliography database. Source links remain pinned, so later changes to `main` do not silently alter the cited code. The delivery PDF was compiled and rendered for visual inspection; build intermediates are excluded from the ZIP.

9 Conclusion

The additional correctness defect is an invalid change of domain in a remainder theorem: real coefficients in a finite approximation were treated as evidence that an uncertain complete function remained on the real line. The public cubic witness makes the failure exact and unambiguous. A small, conservative candidate closes that route without pretending to solve general complex asymptotics.

The additional structural defect is equally localized: alignment combines conditions, and finalization combines one operand again. Repeated one-block arithmetic thereby grows 3^n copies of the same predicates. Structural idempotence restores the intended invariant without discarding assumptions or relying on a solver.

The development contribution turns a classical Euler transform into a narrow, usable negative-Lerch enclosure provider with a proof, explicit endpoint coverage, and exact implementations. Its intervals offer independent checking information rather than merely another expansion produced by the same moment algebra.

The verified scope is Wolfram 15 public observations, focused in-memory edits, and independent reference computations. The remaining Mathics and full-integration work is specified as acceptance work, not described as completed. That separation is essential to making the findings actionable without overstating what the evidence establishes.

A Novelty crosswalk

Item	Nearest earlier record	Excluded earlier mechanism	Additional mechanism here
N01	Review 11, N04[13]	Public routes disagreed about admission of nonreal coefficients.	All stored coefficients are real; an omitted complex tail is amplified and then assigned a false trigonometric magnitude envelope.
N01	Review 37, F01[14]	Sign-based <code>fwdAbs</code> shortcut with retained complex subleading terms.	Composite sine/cosine fallback on a pure remainder; no retained imaginary coefficient is needed.
N01	Review 42, N01[15]	A real absolute-value observable can lose classical derivative regularity.	A magnitude bound fails, rather than a derivative contract; modulus itself remains globally complex-Lipschitz.
N02	Review 47, N01[16]	Duplicate work in mutually recursive Mathics sign/real proof search.	The shared arithmetic operation persists exponentially many condition copies, reproduced in Wolfram.
E01	Review 49, E1[17]	Signed enclosures for positive Lerch parameters.	Negative arguments, positive Euler differences, nested geometric-tail intervals, and the $z = -1$ endpoint.

The exclusion comparison used the registers and nearby originals described in Section 2. None of the above distinctions should be expanded into a claim of exhaustive semantic comparison with every archived file.

B Source coverage record

The ranges below identify the retrieved modular source windows used during investigation. A range is a retrieval scope, not a claim that every definition in it has a new finding. Function names and commit-pinned paths remain the authoritative locators if line presentation changes.

Canonical source	Inspected extent	Purpose in the audit
src/Kernel/AsymptoticAnalysis.wl	Selected ranges 1–315, 390–470, 470–900, 1210–end	Entry/assumption scope, jet precision, real coefficient boundary, forward working order, model/residual and loader paths.
src/Kernel/SeriesOperations.wl	Broad sequential reads through end; some tool displays truncated	Representation, alignment, binary arithmetic, observables, composition, derivative and refinement transport. Finding anchors rechecked in native full source.
src/Kernel/SeriesArithmetic.wl	First 230 lines; long display truncated	Public unary fallback and regular-operand arithmetic dispatch.
src/Kernel/SeriesEnvelopeArithmetic.wl	Sequential reads through end	Composite representation, envelope algebra, unary domain/error theorem.
src/Kernel/DirichletSpecialFunctions.wl	Complete returned module	Zeta/Lerch admission, moments, quantitative remainder formulas.
src/Kernel/InverseCertificates.wl	1–210, 215–265, 260–420	Interval arithmetic, source-domain checks, seed and certificate scope; no new certificate finding asserted.
src/Kernel/GammaForward.wl and GammaInverseOperations.wl	Returned module texts	Product logarithms, exactness and inverse-observable transport.
src/Kernel/SourceCoordinates.wl	1–190; long display truncated	Source reconstruction and replay context.
src/Kernel/FourierCoefficients.wl	1–200; long display truncated	Fourier coefficient algebra and constructor budgets.
src/Kernel/ReciprocalLogOperations.wl	Complete returned module	Specialized composition/derivative contracts.
src/Kernel/MathicsCompatibility.wl, MathicsAssumptions.wl, MathicsTaylor.wl, MathicsCoreFunctions.wl, MathicsNumerical.wl	Returned module texts	Host compatibility boundaries, proof and numerical helpers. Mathics not executed.

Complete-text native loading supplied the actual runtime source for the successful probes, including sections beyond the manually inspected windows. No full local checkout, exhaustive test-source audit, or full CI execution is implied by this coverage record.

C Evidence inventory

Artifact	Meaning
evidence/python_tests.log	Actual 17-method Python run; independent reference and synthetic patch mechanics.
evidence/lerch_grid.json	700 exact interval records, regenerated by the Python suite.
evidence/counterexample_exact_bounds.json	Exact rational values illustrating the diverging lower bound in N01.
evidence/wolfram_observations.md	Manually transcribed native results and excluded infrastructure attempts.
evidence/*_connector_input.wl	Successful baseline, metadata, candidate, and independent-reference native inputs.
patches/edits.json	Three exact replacement specifications, targeting two canonical modules.
tests/patch_acceptance.wlt	Two desired-contract tests; not executed as a file-based suite.
evidence/audit_manifest.json	Revision, runtime, evidence counts, and explicit execution limitations.

References

- [1] Vladimir Reshetnikov, *Asymptotic*, reviewed snapshot of 10 September 2026, commit identified by the following pinned link. <https://github.com/VladimirReshetnikov/Asymptotic/commit/efa1aee4845a9c35e140963a0333d0c9ec33b05>.
- [2] *External code-review index and six wave indexes*, reviewed snapshot. [external-reports/code-review/README.md](#). Wave directories 1–6 were used as the exclusion map.
- [3] *Code-review development status*, inspected lines 1–380. [docs/development/CODE_REVIEW_STATUS.md](#).
- [4] *Main package and jet engine*. [src/Kernel/AsymptoticAnalysis.wl](#). In particular `forwardJet`, `makeForwardObject`, `realCoefficientRows`, and the module loader.
- [5] *Series operations*. [src/Kernel/SeriesOperations.wl](#). In particular `seriesData`, `seriesAlign`, `seriesBinary`, and `seriesJetApply`.
- [6] *Arithmetic dispatch*. [src/Kernel/SeriesArithmetic.wl](#). In particular `seriesArithmeticUnary` and `seriesArithmeticFallbackQ`.
- [7] *Composite envelope arithmetic*. [src/Kernel/SeriesEnvelopeArithmetic.wl](#). In particular `seriesEnvelopeData`, `seriesEnvelopeLimit`, and `seriesEnvelopeUnary`.
- [8] *Dirichlet special-function providers*. [src/Kernel/DirichletSpecialFunctions.wl](#). In particular `dirichletLerchForward` and `dirichletLerchBoundConstant`.
- [9] *Inverse certificates*. [src/Kernel/InverseCertificates.wl](#). The inspected source distinguishes explicit-function interval certification from unspecified input remainders.
- [10] *Mathics compatibility layer*. [src/Kernel/MathicsCompatibility.wl](#).
- [11] *Mathics assumption helpers*. [src/Kernel/MathicsAssumptions.wl](#).
- [12] *Mathics numerical adapters*. [src/Kernel/MathicsNumerical.wl](#).

- [13] *Review 11: additional contract audit*, retained README, especially N04. [external-reports/code-review/wave-2/code-review-11/README.md](#).
- [14] *Review 37: differential code audit*, retained README, especially F01. [external-reports/code-review/wave-5/code-review-37/README.md](#).
- [15] *Review 42: regularity contracts and structural work*, retained README, especially N01. [external-reports/code-review/wave-5/code-review-42/README.md](#).
- [16] *Review 47: incremental technical audit*, retained README, especially Mathics proof-search duplication. [external-reports/code-review/wave-6/code-review-47/README.md](#).
- [17] *Review 49: incremental contract audit*, retained README, especially E1's positive-parameter Lerch enclosure. [external-reports/code-review/wave-6/code-review-49/README.md](#).
- [18] NIST Digital Library of Mathematical Functions, §3.9(ii), *Euler's Transformation*, including the classical alternating-series transform and logarithm example. <https://dlmf.nist.gov/3.9.ii>. Consulted 10 September 2026.
- [19] NIST Digital Library of Mathematical Functions, §25.14, *Lerch's Transcendent*, defining series and integral representations. <https://dlmf.nist.gov/25.14>. Consulted 10 September 2026.
- [20] Wolfram Research, *Sin*, Wolfram Language documentation, complex-valued function definition. <https://reference.wolfram.com/language/ref/Sin.html>. Consulted 10 September 2026.
- [21] Wolfram Research, *And*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/And.html>. Consulted 10 September 2026.